

A Runtime and Non-Intrusive Approach to Optimize EDP by Tuning Threads and CPU Frequency for OpenMP Applications

Janaina Schwarzrock^{ID}, Charles C. de Oliveira^{ID}, Marcus Ritt,
Arthur F. Lorenzon^{ID}, and Antonio Carlos S. Beck^{ID}

Abstract—Efficiently exploiting thread-level parallelism has been challenging. Many parallel applications are not sufficiently balanced or CPU-bound to take advantage of the increasing number of cores and the highest possible operating frequency. Moreover, many variables may change according to the system (input set, microarchitecture, and number of cores) or during execution, influencing each parallel region in different ways. Therefore, the task of rightly choosing the ideal configuration (number of threads and DVFS) for each parallel region to deliver the best Energy-Delay Product (EDP) is not straightforward. While the significant number of variables prevents the use of exhaustive search methods, the changing nature of the problem precludes offline strategies. Few solutions are online and synergistically consider thread throttling and DVFS. However, they lack transparency (demand changes in the original code) and/or adaptability (do not automatically adjust to applications at run-time). Our proposed Hoder covers all the characteristics above, optimizing at run-time any dynamically linked OpenMP application, without requiring any code transformation or recompilation. We show Hoder's efficiency by comparing it to two exhaustive offline and two online search approaches, three state-of-the-art techniques, and regular OpenMP execution, considering different setups (Intel 44-, 16- and 12-core; AMD 8- and 12-core).

Index Terms—Thread throttling, DVFS, OpenMP applications

1 INTRODUCTION

NEW applications that compute huge amounts of data, such as machine learning, smarter search mechanisms, big data, biomedical, and video/audio recognition, have been pushing multithreaded processing to another level of performance requirements. Power consumption is directly proportional to this increasing availability of data to compute, being expected to significantly grow (up to 100 MW) in the next years [1], rising the costs related to the energy consumption and temperature control.

However, not always activating the maximum possible number of available cores to execute an application will provide the best outcome in energy and performance (represented by the Energy-Delay Product, or EDP). Many aspects prevent linear improvements as the number of active threads increases [2], [3], [4], [5]. The same reasoning applies to DVFS (Dynamic Voltage and Frequency Scaling): using the maximum operating frequency will not always result in the highest EDP improvements, although it will very likely deliver the best performance [6], [7], [8]. This can be observed in

Fig. 3 (which we will further discuss in the next section). In 3 out of 4 applications, the best EDP (darkest squares) is not reached executing with the maximum number of threads at the highest frequency. Fig. 3 also shows another interesting aspect: the need for tuning both threads and DVFS if one wants to reach a near-optimum solution. However, most existent solutions apply Thread Throttling without considering DVFS [3], [9], [10], [11] or vice versa [6], [7], [12], [13]. The few that optimize both lack at least one of these two essential characteristics [14], [15], [16]:

Adaptability - the ability to adapt to different parallel regions in the application and according to parameters that are only known at run-time: number of available cores, microarchitecture, input-set, etc. In such cases, offline solutions will not suffice, since once any of these parameters changes, the analysis must be re-executed. *Transparency* - The need not to change the source or binary codes, which usually requires the programmer to use special APIs, languages or toolchains that may not be widely used. Intel and ARM have been showing that maintaining Instruction Set Architectures (ISAs) compatibility is key to reuse legacy code and preserve traditional programming paradigms and libraries.

Considering this scenario, we propose Hoder. *Hoder is capable of automatically tuning, at run-time, the number of threads and DVFS for each individual parallel region of any OpenMP application binary implemented with parallel loops.* Because of its *adaptability*, it automatically considers any changing variable (microarchitecture, number of cores, input set), converging to a near optimal solution in EDP. As Hoder is built on top of the original OpenMP library, it is completely *transparent* to

- Janaina Schwarzrock, Charles C. de Oliveira, Marcus Ritt, and Antonio Carlos S. Beck are with the Institute of Informatics, Federal University of Rio Grande do Sul, Porto Alegre, RS 90040-060, Brazil. E-mail: {jschwarzrock, ccoliveira, mrpritt, caco}@inf.ufrgs.br.
- Arthur F. Lorenzon is with the Federal University of Pampa, Bagé, RS 96460-000, Brazil. E-mail: aflorenzon@unipampa.edu.br.

Manuscript received 30 June 2020; revised 22 Oct. 2020; accepted 30 Nov. 2020. Date of publication 22 Dec. 2020; date of current version 11 Feb. 2021. (Corresponding author: Janaina Schwarzrock.)

Recommended for acceptance by P. Balaji, J. Zhai, and M. Si.
Digital Object Identifier no. 10.1109/TPDS.2020.3046537

both designer and end-user: existing OpenMP applications do not need to be annotated, recompiled or pass through any code transformation, so any existent binary code can be optimized. Such transparency is achieved by redirecting the calls originally targeted to the dynamically linked OpenMP library to our new Hoder OpenMP library, which is done by simply setting an OS environment variable.

However, this dynamic adaptability level comes with an extra cost, which is the learning overhead. As an example, let us consider an application with four parallel regions running on a system that supports 24 threads and has 13 different CPU frequency levels. An exhaustive search would need to test $(24 \times 13)^4$ (= nearly 9.5 billion) configurations, which would have to be repeated for every different input and system at hand, so it is clearly impractical. Therefore, our main challenges are to reduce the problem size considering the limited amount of time for run-time learning, reach a satisfactory solution for real-life systems and still keeping transparency, so any programmer can use our approach.

For that, we experimented different strategies to narrow down the search space and evaluate the effectiveness of our proposed method: *a) two other online strategies* that uses a Hill-Climbing based approach (widely used in the state-of-art): one that applies Thread Throttling only with DVFS set to ondemand; and a second that applies Thread Throttling and then DVFS. *b) Two offline strategies*, which potentially have unlimited time to find a solution: an exhaustive global search that applies thread throttling and DVFS using the same configuration for every parallel region; and an exhaustive local search that only adapts threads and use the OS ondemand for DVFS, but considers each region individually. From these experiments, we show the convex nature of the problem and how we converged to a Fibonacci-based strategy, which is capable of reaching a satisfactory solution within a fixed amount of steps and behave well even when the number of cores increases (good scalability). Then, we also compare our strategy to the *c) Regular OpenMP execution*, which uses the maximum possible number of active threads and DVFS set to OS ondemand (our baseline in this paper) and *d) Three state-of-the-art techniques*: FDT [3], Varuna [10], and an energy minimization approach [17] that tunes threads and DVFS.

Hoder was evaluated in five multicore systems executing nineteen well-known benchmarks. Considering the geometric mean, Hoder improves EDP by 17, 12, 21.5 percent on Intel platforms (12-cores, 16-core, and 44-core, respectively) and 34.3 and 46.4 percent on AMD platforms (8-core and 12-core, respectively) over the regular OpenMP execution.

The contributions of this paper include:

- An online approach capable of automatically applying Dynamic Concurrency Throttling (DCT) and DVFS to an OpenMP application while maintaining the software transparency. Implemented as a library, it optimizes the EDP of the application without any modifications at run-time using any real machine.
- Experimentally show how the problem size can be reduced so it is possible to be performed online, using a particular heuristic suitable for this problem, and therefore decreasing the training overhead. This is

done by a detailed analysis and a set of experiments that justifies the reasons for our design choices.

- A comprehensive evaluation of the Hoder in five different platforms, comparing it to other online and offline search strategies, regular OpenMP execution and the state-of-art. We also characterize its learning overhead and field of applicability.

This paper is organized as follows: Section 2 discusses the related work. Section 3 presents an experimental analysis of our design space exploration, which motivates our implementation choices. Section 4 describes the search strategies that guided us when developing our approach. Section 5 details Hoder. Section 6 presents the methodology of our experiments. Section 7 presents the experimental evaluation and the comparison with state-of-art while Section 8 draws final considerations.

2 RELATED WORK

We start by discussing techniques that only tune the number of threads. The following approaches present some degree of adaptability, even though they still rely in some sort of offline analysis. Thread Reinforcer [18] works offline and involves the execution of the application multiple times for a short period with a different number of threads to find the ideal configuration. Curtis-Maury *et al.* [19] propose a framework that has an offline phase to profile the parallel execution that is used to adapt the number of threads at runtime. ACTOR [20] changes the number of threads of parallel regions identified by the programmer through the use of artificial neural networks, which are trained off-line.

The following strategies are online (i.e., work totally at run-time). Suleman *et al.* propose the FDT framework [3]. After code transformation of an OpenMP application, FDT samples portions of the application at run-time to estimate the application behavior, which is used to tune the number of threads. The system in Porterfield *et al.* [21] adjusts the number of threads based on on-line measurements of system resource usage. In Thread Tailor [9], a static software tool-chain creates as many threads as can be used by the architecture. At runtime, a Just-in-Time compiler applies thread throttling. Varuna [10] comprises an analytical engine which continuously monitors changes in the system to determine the optimum degree of parallelism; and a manager that regulates the execution to match the degree of parallelism previously defined. LAANT and Aurora [11], [22] transparently adapts the TLP degree of OpenMP applications.

Now we discuss techniques that use DVFS only. Hotta *et al.* [23] propose a power-performance optimization model that adapts the CPU frequency at run-time but relies on an off-line phase. Another hybrid approach is the Pack & Cap [8], which manages the CPU voltage-frequency and use thread affinity (but do not perform thread throttling) to optimize performance within a power budget. Run-time DVFS approaches are discussed next. PART [6] is an automatic power-aware run-time system that adapts the CPU frequency. Next, online and transparent mechanisms are detailed. Rizvandi *et al.* [12] propose a maximum-minimum-frequency DVFS algorithm to reduce the energy consumption. Ge *et al.* [7] present a run-time DVFS scheduler for multicore-based power aware clusters.

The aforementioned works tune whether DVFS or Threads. More complete solutions that consider both are discussed next. An offline approach is proposed by De Sensi [24]. It predicts the number of threads and CPU frequency level that offers the best performance and energy consumption. Hybrid approaches include Li *et al.* [25] and Li *et al.* [26]. In the former, the application is executed once for every combination (thread number and DVFS level), and energy and performance data are collected. Then, optimization mechanisms are applied to find the combination that delivers the best result. Li *et al.* [26] propose a instrumentation library for hybrid MPI/OpenMP applications to optimize each OpenMP region. The library has an off-line phase to train a model that will be used at runtime. Representative examples of online approaches are discussed as follows. OpenMPE [14] is an extension designed for energy management of OpenMP applications, in which the programmers insert new directives in OpenMP code to indicate potential regions to save energy, which requires a particular compiler and runtime system. Nornir [15] is a runtime system that monitors the application execution and adjusts the resources configurations (DVFS, number of threads, and thread placement) in order to satisfy either performance or power consumption requirements. Shafik *et al.* [17] propose an adaptive and scalable energy minimization model for OpenMP programs, which comprises two steps: (i) code notations are inserted by the programmer in the code to enable energy minimization with specified performance requirements; (ii) the runtime system read these requirements and uses this information to guide the energy minimization. *Conductor* [27] is a run-time system that improve performance under a power constraint for MPI/OpenMP applications. It needs code modifications to insert functions into the applications. LIMO [16] is a dynamic system that monitors the application and adapt the execution accordingly. However, it requires hardware modifications and special compiler support.

The following techniques aim to reduce power while improving performance. Sarood *et al.* [28] propose an interpolation scheme that optimizes the number of active nodes and the distribution of power between CPU and memory subsystems. Bhalachandra *et al.* [29] propose a runtime model that considers dynamic duty cycle modulation for MPI applications. PUPIL [30] is a hybrid power capping system that combines HW/SW components to improve the performance of parallel applications. Etinski *et al.* [31] propose a job scheduling policy to maximize performance under a given power budget.

Our Contributions. Considering all the works above, Hoder is the only technique that tunes threads and CPU frequency, and does so with minimal overhead, without the need for any sort of offline analysis. Besides being capable of adapting to the system and application at hand, it works with any C/C++ compiler and OpenMP. It means that the Hoder is totally transparent to the end-user and the software developer, who does not need to make any changes in the source code or even recompile it. However, because this high level of transparency, Hoder is limited to OpenMP applications only. Table 4 of the supplemental material, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TPDS.2020.3046537>, compares Hoder to the most of works here discussed.

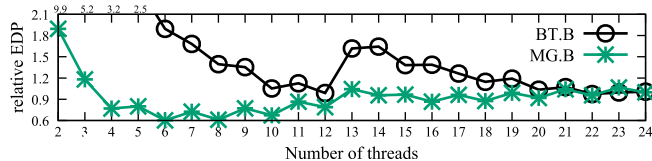


Fig. 1. Relative EDP for BT.B and MG.B running with different #threads.

3 MOTIVATION AND BACKGROUND

3.1 EDP: Energy-Delay Product

In this work, we use EDP as main optimization metric. It has been widely used because it correlates, in a single value, the relationship between energy consumption and performance. For example, considering two scenarios: (i) an application runs in 100 seconds and consumes 10 joules of energy; (ii) an application runs in 50 seconds and consumes 40 joules of energy; scenario (i) would have EDP of 1000, while scenario (ii) would have EDP of 2000. This shows that although scenario (i) is 2 times slower, it has the best trade-off between energy consumption and performance. By adding an exponent x on delay ($EDP = \text{Energy} \times \text{Delay}^x$), it is possible to change the weight of delay (performance) towards energy, which would reflect the importance given to performance considering the field of application.

3.2 Thread Scalability

Many works have shown that selecting the maximum number of available threads (the common choice for most software developers [9]) will not necessarily deliver the best result. The main causes are: (i) instruction issue width saturation in SMT architectures executing CPU-bound applications [2]; (ii) off-chip bus saturation when threads constantly operate on large amounts of data [3]; (iii) data synchronization, which limits the scalability of applications with many synchronization points [3]; and (iv) concurrent shared memory accesses, which affect applications with high communication demands [4]. Fig. 1 (from our own experiments) shows the EDP for two NAS applications: BT and MG, when they run with a different number of threads (x -axis) on a 12-core Intel processor, with SMT enabled and fixed (highest) CPU frequency. Results are normalized to the EDP of the default execution (i.e., using the maximum number of threads available and DVFS governor set to *on-demand*), referred to as the *baseline* (values lower than 1 are better). As one can observe, both applications do not scale as the number of threads increase, which means that in both cases using the maximum number of cores available will not result in the best EDP. Moreover, while the ideal number of threads for the BT (12 or 22) is not significantly better than the baseline, the best configuration for MG (only 6 threads) reduces EDP in 40 percent. This highlights the opportunity of representative gains in certain applications.

3.3 Cooperative Thread Throttling and DVFS

Next we discuss the importance of cooperative optimization of threads and CPU frequency in some cases. A DVFS system takes advantage of the processor idleness (usually provoked by I/O operations or by memory requests) influencing dynamic power and reducing power reduction [32]. However, the OS policy is conservative (always aiming for the

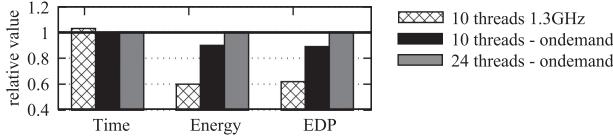


Fig. 2. Jacobi (JA) under different configurations.

best possible performance) and ends up not taking advantage of all optimization opportunities. Let us use the Jacobi benchmark to demonstrate this opportunity window. In Fig. 2, three bars give the results in execution time, energy, and EDP for three configurations: 10 threads @1.3GHz; 10 threads @ondemand and 24 threads @ondemand (which is the default execution and our baseline). Reinforcing the experiment of the previous sub-section, one can observe that the configuration with 10 threads @ondemand has better EDP than the default, since energy consumption drops while performance remains the same. When the application executes with the same number of threads (10) but with a lower CPU frequency (1.3GHz) instead @ondemand, substantial energy gains are achieved at the price of a small performance loss, resulting in significant EDP gains.

3.4 Design Space Opportunities

Considering the previous experiments, in this sub-section we show the huge design space available by considering four benchmarks with distinct behaviors. Fig. 3 illustrates the EDP of FT.C, FFT, MG.B, and Poisson (PO) benchmarks when running on a 12-core (with SMT) with different amounts of threads (x -axis) and CPU frequencies (y -axis). Each square in the matrix represents a configuration given in #Threads operating at a certain frequency. The bar on the right side of each matrix shows the relative EDP (i.e., values are normalized to the baseline, which is represented by value 1), so values lower than 1 mean that a configuration is better than the baseline (standard OpenMP execution). The darker the color, the lower the EDP. Note that EDP scale bar varies depending on the benchmark. Let us first discuss the behavior of the FT.C benchmark (Fig. 3a), a CPU-bound application that presents high opportunities for TLP exploitation. Because of these characteristics, the best EDP will be

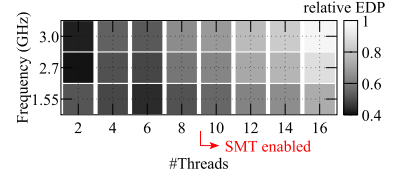


Fig. 4. MG.B on AMD processor. SMT is enabled when #threads > 8.

achieved when it runs with the maximum number of threads and the highest CPU operating frequency, or a number near to it. In such cases, the baseline execution will very likely deliver the best possible result, with no room for further optimization. The FFT benchmark, on the other hand, is an example of application that has thread scalability limited by one of the reasons discussed in Section 3.2. In this case, the off-chip bus saturates, so no further EDP improvements are achieved by increasing the number of threads [3]. Therefore, as one can observe in Fig. 3b, the execution with only four threads delivers the best EDP, which is 23 percent better than the baseline.

MG.B is a memory-intensive application. Because threads communicate by accessing data that are located in shared memory regions, which are usually more distant from the processor (e.g., L3 cache and main memory), they may become a bottleneck. For this kind of application, the best EDP will be very likely be achieved with a lower CPU operating frequency and a small number of threads [7], [19]. In the case of MG.B, the best EDP is with eight threads at 1.8GHz (Fig. 3c). Finally, there are also applications in which the SMT plays a strong role. One can observe this influence by comparing the execution of the PO benchmark (Fig. 3d) with 12 threads to any number of threads from 13 to 24 (meaning that SMT is enabled). In these cases, SMT execution produces an imbalance on the running threads, i.e., some will run using the whole physical core, without SMT; while others will share it. Because the OpenMP synchronization primitive is a barrier, the slow threads will always delay the fast ones (i.e., the threads whose run solo on the cores will have to wait).

3.5 Need for Adaptability and Transparency

When the system changes, the parallel behavior of an application may change as well. To evaluate that, we re-executed the MG application shown in Section 3.4 on an AMD processor. It has a different number of cores (8 with SMT) and only three different DVFS levels available. As one can observe in Fig. 4, the best result is achieved with the minimum amount of threads (two) and intermediate operating frequency. When executing on the Intel processor, two threads would deliver the worst EDP results. Even slightly changes in the microarchitecture (size of instruction queue, reorder buffer, number of issue slots), very common throughout different versions of a processor (e.g.: Intel Core i) may influence the parallel behavior of an application. Significant changes in the system, like in this experiment, would very likely prevent any reuse of previous offline simulations or inferences, so any non-adaptive offline system would have to be reexecuted. Another important aspect that must be considered is how straightforward would be to migrate an application from one environment to another. If one needs to change or instrument the code, costs would

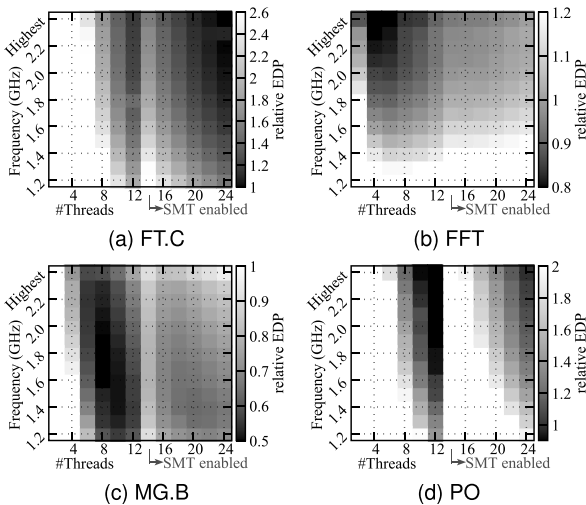


Fig. 3. EDP normalized by the baseline for FT.C, FFT, MG.B, and PO on the 12-core Intel/SMT. SMT is enabled when #threads > 12.

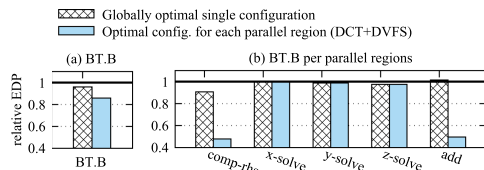


Fig. 5. BT.B on 12-core Intel with SMT.

significantly rise. A transparent technique allows for any architecture migration with zero effort for the developer or end user, no matter the changes in the architecture - we call it *external adaptability*.

So far we have only discussed fixed configurations (i.e., all parallel regions always execute with the very same number of threads at a fixed operating frequency or ondemand during the whole program execution). However, allowing a configuration to change as the program executes (the *internal adaptability*) may enable further gains. To illustrate that, Fig. 5a shows the EDP improvements for the BT benchmark (the baseline is regular OpenMP execution) with internal adaptability, instead of using the best fixed configuration (22 threads at the maximum frequency for all parallel regions). Fig. 5b shows why this improvement is reached: while 3 out of 5 most representative parallel regions that compose the BT have similar behavior and therefore share the best configuration (x-, y- and z- solve, 22 threads @max freq.), 2 of them work at their best with a different configuration. Compute_rhs, for instance, reaches the best EDP with 6 threads at 1.3Ghz.

4 OPTIMIZATION STRATEGIES

We have developed different offline and online search strategies with two objectives: 1) guide us to reduce the search space and 2) so we can assess how efficient our Fibonacci-based approach is and how near to the optimal solutions it can get. Offline strategies can test many configurations without incurring any overhead when executing the application, so their training time can take several hours. However, they need to be re-executed whenever any system parameter changes (e.g., the input size or the microarchitecture). On the other hand, online strategies have the advantage that they can easily adapt to these changes. Nonetheless, such strategies will add extra overhead to the program execution, so the tradeoff between the learning time and quality of solution increases in importance. Hoder uses the Fibonacci online search strategy shown in Section 5.2; and is compared to the ones here described (Section 7), which also gave us directions to its implementation, as discussed in Section 4.3

4.1 Offline Strategies

OPT-Single-Conf. it performs a search for the best configuration (number of threads and CPU frequency) that optimizes the EDP of the whole application when every parallel region runs with the same configuration. Hence, the number of threads and CPU frequency are set at the beginning of application execution and remain the same until the end (even for the sequential regions).

OPT-Set-#Threads. it only considers the number of threads and leaves the OS in charge of the DVFS (using the *ondemand* governor), but finds a particular configuration for

each individual parallel region. As previously mentioned, due to the large design space, an exhaustive search considering the combination of the number of threads of all parallel regions is impractical (i.e., consider all n^k possible solutions, where n is the maximum number of hardware threads and k is the number of parallel regions). Therefore, this search strategy finds the best configuration (number of threads), applying an exhaustive search, for each region individually, so the space exploration does not exponentially grows with the number of parallel regions. Note that this set not necessarily will be the best global solution. The transition from one parallel region to another, which is not taken into account by this strategy, can generate some overhead that creates a dependency between local configurations when creating or destroying threads.

4.2 Online Strategies

We also evaluated two different online strategies, both using a Hill-Climbing based algorithm in their core.

T-only optimizes each parallel region by adapting only the number of threads while using the *ondemand* governor of the OS for DVFS. The search algorithm works as follows: for each parallel region, the search exponentially increases the number of threads (i.e., 2, 4, 8, 16, ...) while there are potential improvements, defining an interval of threads that reduces the size of the space exploration. Then, the algorithm performs a hill-climbing based algorithm with lateral movements in this interval, which is done by testing a neighboring configuration (number of threads) at another point in the search space that has not yet been tested.

T+F seeks for the best number of threads with the operating frequency set to the maximum possible value and, once the number of threads is found, it tests different values for the operating frequency with that fixed number of threads. It uses the same hill-climbing based search strategy used by *T-only* for both threads and operating frequency searches.

4.3 Directions

Based on the experiments presented so far, and aiming to further reduce the search space, we can derive the following *assumptions* from the general behavior of the EDP as a function of the number of threads and the CPU frequency:

Assumption 1. We deal with two axes: number of threads and frequency level. Applying the search for one axis at a time allows us to reduce the search space from (nq) to $(n + q)$, where n is the maximum number of hardware threads and q the number of possible frequency levels. The disadvantage of this reduction is that the best solution may be not achieved in some cases. In contrast, the cost of learning reduces significantly. We will show in our results that in most of cases the reduced cost of learning compensates for the possibility of not finding the ideal solution, even though it still may be representative in some cases. Given that, the number of threads has a stronger influence on the EDP optimization than the CPU frequency, even though tuning operating frequency enables further optimization (Figs. 2, 3c, and 4). Fig. 12 in the supplemental material, available in the online supplemental material, reinforces this statement. It shows the results in EDP and execution time when using an exhaustive search considering a single configuration (best and fixed configuration considering all the parallel

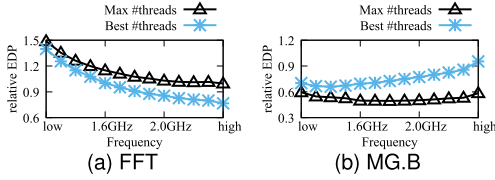


Fig. 6. Convex behavior: FFT and MG.B on the 12-core Intel/SMT.

regions). As one can observe, considering the gmean, optimizing threads brings better benefits than DVFS.

Assumption 2. The experiments performed using the exhaustive algorithms allowed us to characterize the convex nature of the problem. Ignoring local noise (e.g., caused by a slightly different behavior for an odd or even number of threads), the EDP tends to be a convex function in the number of threads (Fig. 1). Moreover, the CPU frequency also tends to be convex. This can be noticed in Fig. 6, which shows the behavior, in terms of EDP, of the CPU frequency levels for the FFT and MG.B benchmarks. Each figure has two curves: one that shows how the CPU frequency behaves with the maximum number of threads active, and another that shows the frequency curve with the best number of threads. In both figures, one can visually verify the convexity of the curve, even though one benefits the most from a high frequency and the other with a lower frequency level. Heuristics such as Hill-Climbing and Fibonacci are suitable for such behaviors. However, as we will show in the results, Hill-Climbing may still take many steps to converge, which inspired us to further decrease the learning overhead.

Assumption 3. Given the aforementioned assumption, another important fact is that our experiments suggest that odd numbers do not bring any significant benefit when comparing to the even ones, regardless of the different scalability behaviors. When considering node-based systems, which is always even, executing odd threads would naturally cause some sort of load imbalance and would very likely not present better results than when comparing with its even ($\#threads - 1$) counterpart. Also, many problems solved in parallel computation are naturally written with even threads in mind (and many times using $\#threads^2$), which also reflects the nature of the input set. In such common cases, using an odd number of threads would unbalance the load across threads. For instance, in the application MG.B, 5 threads delivers a similar or worst EDP than 4 or 6 threads (Fig. 1). Fig. 13 in the supplemental material, available in the online supplemental material, shows such behavior for a large number of applications, reinforcing the use of this assumption.

Assumption 4. Both versions of our experiments (static and dynamic) have shown how SMT influences the results. Let H be the number of physical cores of the system. The EDP as a function of the number of threads is similar in domains $[1, H]$ and $[H + 1, 2H]$ (Fig. 3). Moreover, if the best EDP is in the domain $[H + 1, 2H]$, i.e., using logical threads, the best number of threads tends to be $2H$ (the maximum available threads in the system), as shown in Fig. 3a). From that, we can simplify the thread search space by eliminating the settings between $[H + 1, 2H - 1]$, as there are (i) applications in which enabling SMT provides no gains or even degrade performance (FFT and MG, in Figs. 3b and 3c); (ii) in cases of applications that benefit from SMT (FT.C, in Fig. 3a), the number of threads tends to be the maximum ($2H$); and (iii), even when the best result is outside

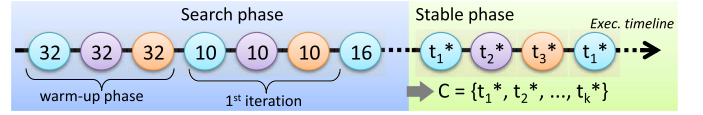


Fig. 7. Search and Stable phases. Each circle represents a parallel region. Circles with the same color represent the same region. The number of threads used to execute a given region is shown in the circle.

the aforementioned space, gains are not representative to compensate the increase in the search space. This last situation can be observed in the BT and PO applications (Figs. 1 and 3d). Fig. 14 in the supplemental material, available in the online supplemental material, depicts how SMT influences performance. It shows the performance speedup relative to the sequential version and the number of idle cycles (average, represented by the bars, and total) as we increase the number of threads for the HS application on the 12-core Intel (24 threads). As we start executing with 13 threads, two will be mapped to the same physical core, activating SMT. From this point on, as the number of threads grows, the average number of idle cycles increases by a small amount or stays constant. However, the total number of idle cycles significantly increases. Because this application has high ILP, there is not enough resources to execute both threads concurrently as if each one was executed on a single core. They become the new critical path of that parallel region, as both threads will delay the execution of the entire parallel region (threads can only synchronize when all have reached the barrier). Therefore, the performance drop is almost recovered only with the maximum number of threads executing.

5 HODER

5.1 Learning and Convergence

Hoder was integrated into the OpenMP library to maintain the software transparency, as described next in Section 5.3. Hoder does not use any information from previous executions. Instead, it converges to a solution and uses it within the same execution of the application at hand. The solution is composed of one configuration ($\#threads$ and CPU frequency) for each individual parallel region from the application, and a given configuration may be different for each region. Whenever the execution enters into a parallel region implemented with parallel loop in any regular executing OpenMP application, Hoder takes action.

Hoder is divided into two phases: learning (i.e., search) and stable, as Fig. 7 illustrates. During the search phase, Hoder employs its learning algorithm over each parallel region. First, in this phase, there is the warm-up step, which is the first time the code of a parallel region executes, provoking cache and TLB misses. These misses significantly degrade performance. Taking this step into account during the convergence would very likely mislead the algorithm to the wrong direction, and that is why Hoder ignores this step for the search, which only starts when TLB and cache misses are stabilized. Hence, in the warm-up step, every parallel region is executed with the default behavior (i.e., maximum number of threads available in the system). After this step, the algorithm starts learning towards the solution (e.g., first iteration in Fig. 7). Every time a parallel region execution is repeated, a new configuration is evaluated, which we call a search step.

For each step, the next configuration is determined by our

customized Fibonacci search strategy, described in Section 5.2. The process of learning is performed to each parallel region individually.

At the end of the search phase each parallel region is assigned with an individual configuration. As the search strategy runs as the application executes, the computation done during this phase is not wasted (i.e., it is used by the application), which helps reducing its costs. Once the search converges to a configuration (i.e., learning process has finished), the stable phase for that parallel region is activated. From this moment on, the particular region will execute with the assigned configuration until the end of the execution. After convergence, Hoder will be responsible for reconfiguring the TLP degree and CPU frequency accordingly each time the execution enters into a parallel region.

5.2 Fibonacci Search Strategy

The main challenge for an online strategy is to reach a satisfactory solution with a low learning overhead. This situation is similar to engineering optimization problems where time or cost imposes strong limits on the number of function evaluations [33], but more difficult, since the cost of each function evaluation, in particular for inefficient parameter settings, contributes to the overall cost, and has to be amortized over subsequent executions. Thus, for regions that are executed only a few times, few probes are essential.

This is the main reason for the following design choices: i) we ignore interaction effects among parallel regions to reduce the search space, ii) we optimize each parameter independently, and iii) we propose a search strategy inspired by the Fibonacci search for continuous domains [34] (adapted for a discrete domain): the solution quality tends to improve and number of probes is less than for other search strategies, since it repeatedly bisects the search interval unevenly to minimize the number of tests.

Considering the assumptions described in Section 4.3, the overall strategy for finding the best configuration is as follows. Based on Assumption 1, Hoder first searches the best number of threads, with a fixed frequency (the highest one); then, it searches for the best frequency level with a fixed thread count (the one selected before). For both the number of threads and frequency levels, Hoder uses a Fibonacci-based search (described in detail below). To filter out small variations (see Assumptions 2 and 3), the search considers only even thread numbers. With respect to the Assumption 4, our approach reduces the search space, eliminating from the search all the configurations using SMT except the one with the maximum number of threads. Since the frequency levels function is already convex (Assumption 2), the Fibonacci search considers all frequency levels.

For simplicity, we assume that the length of the search interval $[l, u]$ is a Fibonacci number F_k (the algorithm can easily be made to work with general search intervals).¹ For a given search interval of length F_k the search probes the objective function at points $t = u - F_{k-1}$ and $t' = l + F_{k-1} - 1$. Note that $t < t'$ for $k > 3$. If the objective function at t is smaller than at t' , the search interval is reduced to $[l, t']$, otherwise to $[t, u]$. After bisection, the length of the reduced

interval is the next smallest Fibonacci number F_{k-1} , and therefore the objective function value of one of the test points in the next iteration is already known, such that only one additional test per iteration is required. To see the latter, observe that when the interval is reduced to $[l, t']$, test point is already at $t = u - F_{k-1} = l + F_k - 1 - F_{k-1} = l + F_{k-2} - 1$, by definition of the Fibonacci numbers. The same holds when reducing to interval to $[t, u]$ for point t' . The search stops when $k = 3$, i.e., for interval size $F_k = 2$, and returns the best value found. For an interval of length $n = F_k$ the search needs at most k probes, such that the number of probes is bounded by $\lceil \log_\varphi(\sqrt{5}n + 1/2) \rceil = \Theta(\log_\varphi n)$, where $\varphi = (1 + \sqrt{5})/2 \approx 1.62$ is the golden ratio.

5.3 Hoder Integration to OpenMP

OpenMP is a parallel programming interface for shared memory in C/C++ and FORTRAN. It usually requires less effort to extract parallelism when compared to other APIs (e.g., PThreads, MPI), making it more appealing to developers. It is widely used and most parallel benchmarks are implemented with it: SPEComp [35], NAS-PB [36], Parboil [37], Rodinia [38], etc. OpenMP provides three ways to exploit parallelism: parallel loops, sections, and tasks. Parallel loops are the most common approach and all benchmarks listed above are implemented in this way.

We built Hoder by incorporating its functions into the *libgomp*, the GNU Multi-Processing Run-time Library responsible for all OpenMP functionalities. It is dynamically linked to applications, so modifications in its code are completely transparent to user applications. Hoder still holds all the original functionalities of OpenMP and may co-exist with the original OpenMP in the system. To use Hoder, the user simply has to replace the original OpenMP *libgomp* with Hoder's *libgomp* (which also includes the original OpenMP functions). When the environment variable *OMP_HODER* is set in the Linux OS, the runtime system of Hoder is used instead of the regular OpenMP functions. Otherwise, the application executes with the original OpenMP. In cases where there are parallel regions implemented using a different API or unsupported OpenMP directives, Hoder will still work for each *OpenMP parallel directive* region and will not influence the execution of those other parallel regions. There is no need to make any modifications in the OS (package installation, kernel recompilation) or having super user permission. Hoder requires an interface to access performance counters for power dissipation and execution time.

We illustrate in Fig. 15 (supplemental material, available in the online supplemental material) how Hoder operates by using an OpenMP code example and the respective *libgomp* functions. Hoder comprises four functions that were included in the original *libgomp*: *initHoder()* initializes the data structures, libraries, and variables used to control the search algorithm (described in Section 5.2). *hoderResolveThreadsDVFS()* sets the number of threads and DVFS to be used for the current parallel region based on the current state of the search algorithm. It also initializes the counters for collecting data from the execution environment of the current parallel region. *hoderEndParallelRegion()* is executed after the execution of the current parallel region to get its EDP. Time is extracted by the OpenMP *omp_get_wtime()*

1. The Fibonacci numbers are defined by $F_0 = 0$, $F_1 = 1$, and $F_k = F_{k-1} + F_{k-2}$ for $k > 1$.

TABLE 1
Evaluated Setups

	8-core AMD Ryzen 7 1700/SMT	12-core AMD Ryzen 9 3900x/SMT	12-core Intel Xeon E5- 2630/SMT	16-core Intel Xeon E5- 2640/SMT	44-core Intel Xeon E5- 2699v4/SMT
#Threads	16	24	24	32	88
Shared L3	16MB	64MB	30MB	40MB	55MB
RAM	16GB	32GB	32GB	64GB	256GB
DVFS	3 levels 1.5-3.0GHz	3 levels 2.2-3.8GHz	13 levels 1.2-2.3GHz	10 levels 1.2-2.0GHz	12 levels 1.2-2.2GHz
#Confs	48	72	312	320	1056

function, while the energy consumption is retrieved from the hardware counters present in modern processors via the Running Average Power Limit (RAPL) library. It provides a set of counters that gives energy and power consumption information by using hardware performance counters, which has already shown to be very accurate [39]. *hoderEndParallelRegion()* also performs one step of the search algorithm if it has not converged yet, defining the pair threads/frequency that will be used for next iteration of the current parallel region. *HoderDestructEnv()* concludes and destroys the Hoder environment at the end of execution.

6 METHODOLOGY

Platforms. The experiments were performed on five platforms with Linux 4.15.0 (Table 1). The last row (#confs) depicts the number of possible configurations supported by each processor (#threads $\times$ #DVFS levels) – however, it is important to highlight that each parallel region have its own configuration, which may be different from the configuration of the other regions, so the design space exploration grows exponentially with the number of parallel regions.

Benchmarks. We have used nineteen OpenMP applications written in C/C++. Seven kernels from the OpenMP-C version of NAS-PB [36]: BT, CG, FT, IS, LU, MG, and SP. Two applications from the Rodinia Benchmark Suite [38]: *hotspot* (HS) and *streamcluster* (SC); Five applications from different domains: *Fast Fourier Transform* (FFT) [40], *Jacobi* (JA) [41], *Poisson* (PO) [41], *STREAM* (ST) [42], and the *high performance conjugate gradient* benchmark (HPCG) [43]; and five applications from the Parboil Benchmark suite [37]: CUTCP, LBM, MM, SPMV, and STENCIL.

They were executed with the following inputs: the NAS-PB with the std input sets, i.e., “B” or “C”; HS with a square matrix of 1024 and 200k iterations; SC with a dimension of 512 and 65536 points; FFT with an array of 10k and 25 iterations (FFT has workload variability, that is, as it executes, the size of the data structure and the number of iterations change); Jacobi with a square matrix of 8192 and 1500 iterations; Poisson with a square matrix of 768 and tolerance of 10^{-6} ; ST with an array of size 40M and 1k iterations; HPCG with dimensions of $256 \times 256 \times 128$; and the Parboil benchmarks with the largest dataset available for each benchmark. To ensure the application execution correctness, we compared the outputs of the executions with Hoder enabled to the execution of their original counterpart.

We bind threads to cores using the Scatter policy to ensure load balancing between the NUMA nodes (i.e., it distributes the threads as evenly as possible between the

nodes). For memory page allocation, we use the standard Linux’s First-Touch policy (although not guaranteed, it weights the location of access when mapping data). We compiled each application with GNU GCC/G++ 8.3 and OpenMP 4.5, with the optimization flag -O3. We executed each experiment ten times (standard deviation of 1 percent).

Table 5 in the Supplemental Material, available in the online supplemental material, characterizes the applications in terms of IPC (instructions per cycle), TLP, and communication rate. IPC somehow reflects the available instruction level parallelism of the application and how CPU- or Memory bound each application is. The higher the IPC, more pressure is put on the microarchitecture (and mainly SMT), which may lead to issue-width saturation in the processor (one of well-known bottlenecks in the literature). Higher IPC also tends to decrease the waiting time of unbalanced threads (and therefore decrease some of opportunities of optimization). Applications with low IPC and high communication demands have several data dependencies that must be addressed at runtime to ensure correctness of the results. Consequently, they demand large amounts of communication among threads which may lead to off-chip bus and shared memory saturation; as well as to unbalanced execution because of extra data synchronization needed. Finally, TLP gives us the thread level parallelism of an application as defined by Blake *et al.* [44]: the average amount of concurrency exhibited by the program during its execution when at least one core is active. The closer this value is to 1.0 (normalized to the total number of cores available), the more TLP is available. With this set, we cover a wide range of behaviors w.r.t. to TLP, ILP and its level of communication of applications.

7 EXPERIMENTAL EVALUATION

7.1 Hoder Versus Online Strategies and Baseline

Figs. 8a and 8b presents the geometric mean (*gmean*) of EDP and Execution time considering the five multicore systems.² Fig. 9 details the EDP results for the entire benchmark set for the 44-core Intel (88 threads) and 12-core AMD (24 threads). Results are normalized to the regular OpenMP Execution (baseline, represented by the black line).

Hoder versus Baseline. In most cases, Hoder shows EDP improvements over the regular OpenMP execution (Figs. 8 and 9). Hoder achieves EDP gains of 21.5, 12, and 17.2 percent on the Intel platforms (44-, 16-, and 12-core, respectively); 46.4 and 34.3 percent on the AMD platforms (12- and 8-core, respectively), when considering the *geomean* of the entire benchmark set on each processor (Fig. 8a). Also, as observed in (Fig. 8b), there are no significant performance losses for the Intel systems (therefore the EDP gains come from the energy savings); while for the the AMD system Hoder improves performance (18.6 and 6.8 percent on the AMD platforms 12- and 8-core, respectively). Table 6 of the supplemental material, available in the online supplemental material, details this data.

Considering the best case for Hoder (SP benchmark on all platform), EDP was reduced by 74.8 percent on the Intel 44-core system (Fig. 9a) and 86.7 percent on the AMD 12-core

2. All the results for the five platforms are shown in Figs. 16, 17, and 18, available in the online supplemental material. Fig. 22, available in the online supplemental material, shows results for the FFT, JA, PO, HS, HPCG, ST applications only.

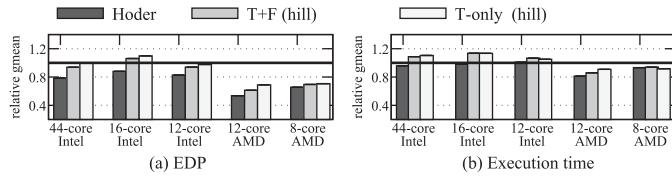


Fig. 8. Results (geometric mean, normalized to the baseline) in EDP and Execution time for Hoder and online strategies on all systems.

system (Fig. 9b); In this particular case, a significant performance improvement contributed to these results (42.8 percent on the Intel 44-core and 56.2 percent on the AMD 12-core). When considering the overall geometric mean (entire benchmark set and all processors), Hoder provided 27.4 percent of EDP improvements. In very specific scenarios where the DSE is limited (e.g., the best EDP is achieved with the maximum number of threads running at the highest operating frequency), it presents similar or worst results as the baseline. We discuss this behavior in more details in Section 7.2.

We also evaluate Hoder with ED²P as the optimization objective – which gives more weight to delay– on our two most complex platforms: 44-core Intel and 12-core AMD (Figs. 19 and 20 of the supplemental material, available in the online supplemental material). When targeting to ED²P, Hoder still presents significant gains in EDP and ED²P; but present deliver further performance improvements when comparing to when the algorithm targets EDP only. Finally, we also evaluate Hoder with one knob disabled (i.e., Hoder adapting either Threads or CPU frequency) on our 44-core Intel platform. We present these results in Fig. 21 of the supplemental material, available in the online supplemental material, which reinforces the need to apply both together.

Hoder versus T+F and T-only. Hoder performed better than the hill-climbing strategies (T+F and T-only) in most cases. Comparing the geometric mean of the results on the Intel processor, Hoder achieved EDP gains of 16 and 22 percent for T+F and T-only (44-core), 17 and 20 percent for T+F and T-only (16-core), and 12 and 15 percent for T+F and T-only (12-core). When considering the AMD processor, the gains are of 13 and 22 percent for T+F and T-only (12-core), and 5.6 and 7 percent for T+F and T-only (8-core). The following observations can be done: (i) Optimizing operating frequency enables further gains, as one can observe when comparing both hill-climbing based strategies: T-Only and T+ F; (ii) Hoder is more efficient than those strategies. Fibonacci

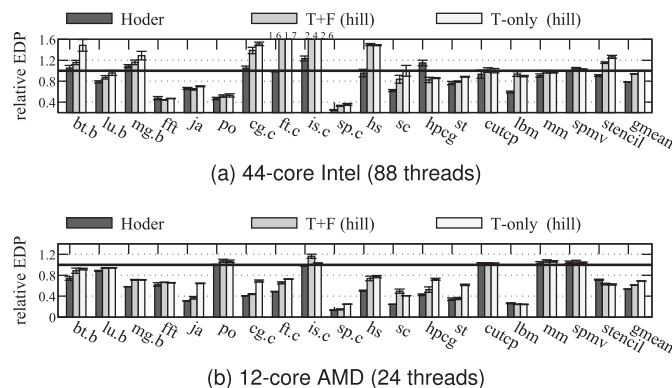


Fig. 9. Results in EDP (normalized to the baseline) for Hoder and online strategies on 44-core Intel and 12-core AMD for the entire bench. set.

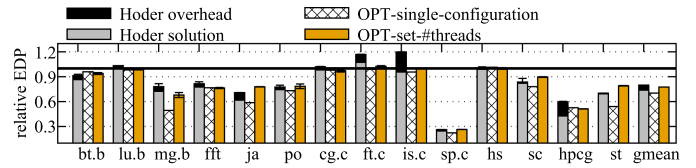


Fig. 10. Hoder versus Offline strategies executing on the 12-core Intel.

based search is capable of finding a better final solution with fewer search steps; (iii) The more extensive the exploration space, the better are the gains achieved by Hoder. This statement can be observed by comparing the results between the 44-core Intel (1056 configurations per parallel region) and 8-core AMD (only 48 configurations per parallel region). It means that Hoder is scalable, that is, even when the space exploration increases, it is capable of converging to a satisfactory solution within a reasonable amount of time.

7.2 Effectiveness and Costs of Hoder

7.2.1 Hoder Overhead

To measure the overhead of our Fibonacci search, we first executed the applications with Hoder enabled and saved the solution found for each parallel region. Then, we reran the applications with the solution previously found without the Fibonacci search. In this second execution round (referred as the *Hoder Solution*), Hoder was only responsible for setting the number of threads and DVFS for each parallel region according to what was found in the first execution. The difference between these two executions results in the *Hoder Search Overhead*. Fig. 10 shows the Hoder search overhead for each benchmark (left-most bar, represented in black color) running on the 12-Core Intel processor.³ This overhead may be significant and the main reason why EDPs get worst than the original baseline in some cases.

The overhead is originated from two situations: (i) The execution of the search algorithm itself. The slowest part is reading the hardware counters (0.00033s for each iteration). However, considering that Hoder performs few search steps, this cost is not representative in the end (and cannot be observed in the Figure); and (ii) The execution of a given parallel region with a configuration that is not the ideal, while the search algorithm is trying different possibilities to converge. Fig. 11 characterizes these costs, and shows that the process of searching for the right number of threads is the most expensive in EDP than the process of finding the CPU frequency. There are two reasons for this: 1) as we have already discussed, threads plays a major role for the EDP than the CPU frequency; and 2) when Hoder is searching for the number of threads, it does not necessarily use the best operating frequency (in this phase, the operating frequency is set to the maximum). On the other hand, when Hoder is tuning the operating frequency, the parallel region is already executing with a potentially near-optimum result in terms of threads, naturally decreasing the overhead.

Therefore, the quality of the intermediate configurations w.r.t. to the optimal is key. Let us consider the IS benchmark: any configuration that is not near to the maximum

3. Fig. 23 of the supplemental material, available in the online supplemental material, shows Hoder search overhead in terms of execution time, which is similar to EDP.

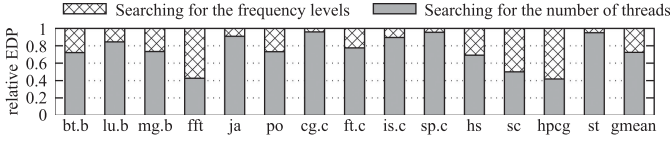


Fig. 11. Search overhead characterization.

number of threads or operating frequency will add extra overhead in terms of EDP. Applications like SP, on the other hand, have many configurations that are already better than the baseline, so gains are already achieved before convergence. Hence, depending on the costs of not executing a given parallel region with the near-optimum solution, there may have overheads even when few steps are necessary for convergence. Such costs, however, tend to be amortized if the application executes for more time.

Table 2 gives more details on the search process, depicting the following data: (i) the *number of significant parallel regions* of each benchmark (the most executed ones, no matter the size); (ii) the total *amount of steps* taken by Hoder during the learning phase (i.e., the number of iterations). They are classified as the steps necessary to tune the number of threads and operating frequency. For instance, BT has 5 parallel regions, and each one needed 5 steps to converge to the right amount of threads, and 5 more steps for the operating frequency. (iii) The *amount of iterations of each parallel region* necessary for the searching process w.r.t. to the total number of iterations of that region during the program execution. As one can observe, the benchmarks FT and IS have a searching phase that is very significant w.r.t. to the total execution time, which impacts the final result; and (iv) how many *distinct configurations per application*. It shows that in most cases Hoder found different configurations for different parallel regions, highlighting the importance of internal adaptability. The benchmarks that have only one configuration are usually those which the best configuration is near to the baseline (maximum number of threads at the highest frequency).

7.2.2 Quality of Solution

To assess the efficiency of the heuristic implemented into Hoder, we compare the EDP results of *Hoder Solution* to the ones achieved by the offline strategies discussed in Section 4.1 (*OPT-set-#threads* and *OPT-single-conf*). By ignoring its online costs (i.e., not considering the overhead of the learning phase), we can isolate the solution found by Hoder and compare it to the Offline strategies. They are presented in Fig. 10 and are normalized to the baseline. In most cases, Hoder Solution was able to achieve similar results as the offline approaches. When comparing the geometric mean of the entire benchmark set, Hoder Solution achieved an EDP 5 percent better than *OPT-set-#threads* and is only 5 percent distant from the result found by the *OPT-single-conf*.⁴

It is natural that the offline strategies perform better than Hoder in some cases, since they test all possible configurations according to their limitations. Hoder performs worst than *OPT-single-conf* because of two main reasons: (i) the solution found by Hoder is not the ideal one and the

TABLE 2
Hoder Statistics: 12-Core Intel

BT	LU	MG	FFT	JA	PO	CG	FT	IS	SP	HS	SC	HPCG	ST
# parallel regions													
5	2	6	3	1	4	3	5	1	9	1	2	4	1
# search steps (#threads)													
25	13	35	15	5	19	9	27	4	45	4	8	16	4
# search steps (CPU frequency)													
25	10	30	15	5	5	5	20	5	45	5	10	20	5
% of iterations in the searching phase w.r.t total execution													
4.9	4.4	5.1	0.005	0.6	0.9	6.0	42	60	2.5	0.005	0.35	3.01	0.90
# distinct configurations (#threads and frequency)													
3	2	6	2	1	2	3	2	1	6	1	2	4	1

benchmark has a more constant behavior in terms of configurations; (ii) the associated costs of changing the number of threads and operating frequency from one parallel region to another at runtime (e.g., creation, termination, context switching). The *OPT-single-conf* uses a fixed configuration for parallel regions throughout program execution, so the overhead involved in changing configurations is only observed when switching from a parallel to a sequential region and vice-versa, when the configurations are different. In this sense, Hoder is more similar to *OPT-set-#threads*, since it also optimizes locally (so configurations between parallel regions also may change). Therefore, *OPT-set-#threads* is also more affected by the switching overhead, but at different rates (MG, JA, PO, FT, IS, SP, SC, and ST). The advantage of Hoder over the *OPT-set-#threads* is that, because it is online, some of the switching overheads can be caught and automatically taken into account during the learning process, making it to converge to a different (and sometimes better) configuration. Hoder Solution outperformed *OPT-set-#threads* in BT, JA, PO, IS, SP, SC, HPCG and ST benchmarks.

It is important to point out that Hoder, because of its dynamic nature, is unable to use a fixed configuration as its main strategy, since it cannot guarantee that it will consider all parallel regions during the learning phase. For instance, let us consider a very common behavior: the first half of the application code executes and only after that the other half begins. Dynamically considering all the parallel regions and testing them with a fixed configuration would be impossible in such cases. Another important aspect is that dynamic techniques are unable to measure how representative the parallel region is during the learning process (i.e., how many iterations it will have during execution).

7.3 Hoder Versus State-of-the-Art

We compared Hoder to three well-known approaches: FDT [3] and Varuna-PM [10] that tune the number of threads and the approach proposed by Shafik *et al.* [17] that tune both TLP and DVFS (discussed in Section 2). We have faithfully implemented the FDT mechanism and the method proposed by Shafik *et al.* into the OpenMP code. In the same way, we implemented the programming model employed by Varuna and applied it to our benchmarks. Table 3 depicts the EDP results of each approach (Hoder, FDT, and Varuna) normalized to the Regular OpenMP execution. As observed in Table 3, Hoder outperforms them.⁵ We discuss the reasons below.

4. Fig. 23 of the supplemental material, available in the online supplemental material, shows that Hoder solution achieved a performance improvement only 1 and 2 percent distant from the result found by *OPT-set-#threads* and *OPT-single-conf*.

5. Table 7 of the supplemental material, available in the online supplemental material, shows execution times.

TABLE 3
EDP: Hoder, FDT, Varuna-PM, and Shafik *et al.*

	bt.b	lu.b	mg.b	fft	ja	po	cg.c	ft.c	is.c	sp.c	hs	sc	hpcg	st	gmean
Hoder	0.9	1.0	0.8	0.8	0.7	0.8	1.0	1.2	1.2	0.3	1.0	0.8	0.6	0.7	0.8
FDT	1.2	1.5	2.8	14.0	1.7	0.7	2.7	5.4	7.0	0.6	1.0	1.1	1.3	1.1	1.9
Varuna	1.5	1974	18.1	2113	1.6	404	4.9	1.9	155	0.6	449	1.9	0.9	1.3	15.8
Shafik	1.1	2.1	1.9	19.8	0.9	2.0	2.0	5.1	19.4	0.5	22.9	0.8	0.4	0.9	2.3

FDT ignores fundamental hardware characteristics, converging to a non-optimal number of threads in many times. Moreover, the training phase of FDT executes each parallel region in single threaded mode until the standard deviation of the observed metric (memory bandwidth usage or synchronization time) is stable. It leads to a higher overhead for applications that present high TLP (e.g., CG). Because of this, in many cases FDT is worse than the baseline.

Varuna-PM was developed to be used in different kinds of applications (e.g., big data and ones that are recursively implemented), since it creates as many threads as possible (we executed each application with 1566 threads - numbers taken from [10]). Moreover, it targets performance rather than EDP. It is important to emphasize that these results do not consider the improvements provided by the analytic engine and the manager system of Varuna. However, even if they could improve performance by 15 percent and reduce energy consumption by 31 percent (values taken from [10]), it would not be enough to provide the same EDP levels as Hoder.

Shafik *et al.* [17] aims at minimizing energy given a power budget defined by the user (we evaluated different power budgets, ranging from 80W to 140W, and considered the best result). Because its search algorithm starts the execution with two threads and increases the number of threads by one while there is still room to increase the processor power, the proposed approach can achieve better results than the baseline in applications with low TLP (SP and JA). However, the learning overhead (because the way the search is performed) is significant for applications with medium and high degree of TLP (e.g., BT and CG).

8 CONCLUSION

Hoder is an approach capable of automatically finding, at run-time, the optimal number of threads and CPU operating frequency for each parallel region. It is completely transparent to both designer and end user: given an OpenMP application binary, Hoder optimizes it without any code changes, transformation or recompilation. As future work, we will include different policies for thread and data mapping. Considering that they will significantly increase the design space, we will have to evaluate the use new strategies and exploit constant learning throughout different executions of the application. Therefore, we also plan to decrease the search overhead (such as stopping the convergence phase earlier for some parallel regions), and further exploit current hardware characteristics to improve the algorithm w.r.t. its starting search point (helping it to converge faster).

ACKNOWLEDGMENTS

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior—Brasil (CAPES) - Finance Code 001, FAPERGS and CNPq.

REFERENCES

- [1] P.-F. Dutot, Y. Georgiou, D. Glesser, L. Lefevre, M. Poquet, and I. Rais, "Towards energy budget control in HPC," in *Proc. 17th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2017, pp. 381–390.
- [2] S. E. Raasch and S. K. Reinhardt, "The impact of resource partitioning on SMT processors," in *Proc. 12th Int. Conf. Parallel Architectures Compilation Techn.*, 2003, pp. 15–25.
- [3] M. A. Suleman, M. K. Qureshi, and Y. N. Patt, "Feedback-driven threading: Power-efficient and high-performance execution of multi-threaded workloads on CMPs," *SIGARCH Comput. Archit. News*, vol. 36, no. 1, pp. 277–286, 2008.
- [4] L. Subramanian, V. Seshadri, Y. Kim, B. Jaiyen, and O. Mutlu, "MISE: Providing performance predictability and improving fairness in shared main memory systems," in *Proc. IEEE 19th Int. Symp. High Perform. Comput. Archit.*, 2013, pp. 639–650.
- [5] A. F. Lorenzon and A. C. S. Beck, *Parallel Computing Hits the Power Wall—Principles, Challenges, and a Survey of Solutions*. Berlin, Germany: Springer, 2019.
- [6] C. Hsing Hsu and W. Chun Feng, "A power-aware run-time system for high-performance computing," in *Proc. ACM/IEEE Conf. Supercomputing*, 2005, pp. 1–1.
- [7] R. Ge, X. Feng, W. Feng, and K. W. Cameron, "CPU MISER: A performance-directed, run-time system for power-aware clusters," in *Proc. Int. Conf. Parallel Process.*, 2007, pp. 18–18.
- [8] R. Cochran, C. Hankendi, A. K. Coskun, and S. Reda, "Pack & cap: Adaptive DVFS and thread packing under power caps," in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2011, pp. 175–185.
- [9] J. Lee, H. Wu, M. Ravichandran, and N. Clark, "Thread tailor: Dynamically weaving threads together for efficient, adaptive parallel applications," *SIGARCH Comput. Archit. News*, vol. 38, no. 3, pp. 270–279, 2010.
- [10] S. Sridharan, G. Gupta, and G. S. Sohi, "Adaptive, efficient, parallel execution of parallel programs," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2014, pp. 169–180.
- [11] A. F. Lorenzon, C. C. D. Oliveira, J. D. Souza, and A. C. S. B. Filho, "Aurora: Seamless optimization of OpenMP applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 5, pp. 1007–1021, May 2019.
- [12] N. B. Rizvandi, J. Taheri, A. Y. Zomaya, and Y. C. Lee, "Linear combinations of DVFS-enabled processor frequencies to modify the energy-aware scheduling algorithms," in *Proc. 10th IEEE/ACM Int. Conf. Cluster Cloud Grid Comput.*, 2010, pp. 388–397.
- [13] Y. L. Chen, M. F. Chang, W. Y. Liang, and C. H. Lee, "Performance and energy efficient dynamic voltage and frequency scaling scheme for multicore embedded system," in *Proc. IEEE Int. Conf. Consum. Electron.*, 2016, pp. 58–59.
- [14] F. Alessi, P. Thoman, G. Georgakoudis, T. Fahringer, and D. S. Nikolopoulos, *Application-Level Energy Awareness for OpenMP*. Cham, Switzerland: Springer, 2015, pp. 219–232.
- [15] D. D. Sensi, M. Torquati, and M. Danelutto, "A reconfiguration algorithm for power-aware parallel applications," *ACM Trans. Archit. Code Optim.*, vol. 13, no. 4, pp. 43:1–43:25, 2016.
- [16] G. Chadha, S. Mahlke, and S. Narayanasamy, "When less is more (LIMO): Controlled parallelism for improved efficiency," in *Proc. Int. Conf. Compilers Architectures Synthesis Embedded Syst.*, 2012, pp. 141–150.
- [17] R. A. Shafik, A. Das, S. Yang, G. Merrett, and B. M. Al-Hashimi, "Adaptive energy minimization of OpenMP parallel applications on many-core systems," in *Proc. 6th Workshop Parallel Program. Run-Time Manage. Techn. Many-Core Architectures*, 2015, pp. 19–24.
- [18] K. K. Pusukuri, R. Gupta, and L. N. Bhuyan, "Thread reinforcer: Dynamically determining number of threads via OS level monitoring," in *Proc. IEEE Int. Symp. Workload Characterization*, 2011, pp. 116–125.
- [19] M. Curtis-Maury, J. Dzierwa, C. D. Antonopoulos, and D. S. Nikolopoulos, "Online power-performance adaptation of multithreaded programs using hardware event-based prediction," in *Proc. 20th Annu. Int. Conf. Supercomputing*, 2006, pp. 157–166.
- [20] M. Curtis-Maury, F. Blagojevic, C. D. Antonopoulos, and D. S. Nikolopoulos, "Prediction-based power-performance adaptation of multithreaded scientific codes," *IEEE Trans. Parallel Distrib. Syst.*, vol. 19, no. 10, pp. 1396–1410, Oct. 2008.
- [21] A. K. Porterfield, S. L. Olivier, S. Bhalachandra, and J. F. Prins, "Power measurement and concurrency throttling for energy reduction in OpenMP programs," in *Proc. IEEE Int. Symp. Parallel Distrib. Process. Workshops Phd Forum*, 2013, pp. 884–891.

- [22] A. F. Lorenzon, J. D. Souza, and A. C. S. Beck, "LAANT: A library to automatically optimize EDP for OpenMP applications," in *Proc. Des. Autom. Test Eur. Conf. Exhibit.*, 2017, pp. 1229–1232.
- [23] Y. Hotta, M. Sato, H. Kimura, S. Matsuoka, T. Boku, and D. Takahashi, "Profile-based optimization of power performance by using dynamic voltage scaling on a PC cluster," in *Proc. 20th IEEE Int. Parallel Distrib. Process. Symp.*, 2006, pp. 1–8.
- [24] D. De Sensi, "Predicting performance and power consumption of parallel applications," in *Proc. 24th Euromicro Int. Conf. Parallel Distrib. Netw.-Based Process.*, 2016, pp. 200–207.
- [25] J. Li and J. F. Martinez, "Dynamic power-performance adaptation of parallel computation on chip multiprocessors," in *Proc. 12th Int. Symp. High-Perform. Comput. Archit.*, 2006, pp. 77–87.
- [26] D. Li, B. R. de Supinski, M. Schulz, K. Cameron, and D. S. Nikolopoulos, "Hybrid MPI/OpenMP power-aware computing," in *Proc. IEEE Int. Symp. Parallel Distrib. Process.*, 2010, pp. 1–12.
- [27] A. Marathe, P. E. Bailey, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "A run-time system for power-constrained HPC applications," in *High Performance Computing*, J. M. Kunkel and T. Ludwig, Eds. Cham, Switzerland: Springer, 2015, pp. 394–408.
- [28] O. Sarood, A. Langer, L. Kalé, B. Rountree, and B. de Supinski, "Optimizing power allocation to CPU and memory subsystems in overprovisioned HPC systems," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2013, pp. 1–8.
- [29] S. Bhalachandra, A. Porterfield, and J. F. Prins, "Using dynamic duty cycle modulation to improve energy efficiency in high performance computing," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshop*, 2015, pp. 911–918.
- [30] H. Zhang and H. Hoffmann, "Maximizing performance under a power cap: A comparison of hardware, software, and hybrid techniques," *SIGARCH Comput. Archit. News*, vol. 44, no. 2, Mar. 2016, pp. 545–559.
- [31] M. Etinski, J. Corbalan, J. Labarta, and M. Valero, "Optimizing job performance under a given power constraint in HPC centers," in *Proc. Int. Conf. Green Comput.*, 2010, pp. 257–267.
- [32] H. Esmailzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, "Dark silicon and the end of multicore scaling," in *Proc. 38th Annu. Int. Symp. Comput. Archit.*, 2011, pp. 365–376.
- [33] D. R. Jones, M. Scholau, and W. J. Welch, "Efficient global optimization of expensive black-box," *J. Global Opt.*, vol. 13, pp. 455–492, 1998.
- [34] J. Kiefer, "Sequential minimax search for a maximum," *Proc. Amer. Math. Soc.*, vol. 4, pp. 502–506, 1953.
- [35] V. Aslot, M. Domeika, R. Eigenmann, G. Gaertner, W. B. Jones, and B. Parady, *SPEComp: A New Benchmark Suite for Measuring Parallel Computer Performance*. Berlin, Germany: Springer, 2001, pp. 1–10.
- [36] S. Seo, G. Jo, and J. Lee, "Performance characterization of the NAS parallel benchmarks in OpenCL," in *Proc. IEEE Int. Symp. Workload Characterization*, 2011, pp. 137–148.
- [37] J. Stratton *et al.*, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," Center Reliable High-Perform. Comput., Univ. Illinois, Tech. Rep. IMPACT-12-01, 2012.
- [38] S. Che *et al.*, "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE Int. Symp. Workload Characterization*, 2009, pp. 44–54.
- [39] M. Hähnel, B. Döbel, M. Völpe, and H. Härtig, "Measuring energy consumption for short code paths using RAPL," *SIGMETRICS Perform. Eval. Rev.*, vol. 40, no. 3, pp. 13–17, 2012.
- [40] W. Petersen and P. Arbenz, *Introduction to Parallel Computing: A Practical Guide With Examples* in C. Oxford, U. K.: Oxford Univ. Press, 2004.
- [41] M. Quinn, *Parallel Programming in C With MPI and OpenMP*. New York, NY, USA: McGraw-Hill, 2004.
- [42] J. D. McCalpin, "Memory bandwidth and machine balance in current high performance computers," *IEEE Comput. Soc. Tech. Committee Comput. Archit. Newsletter*, pp. 19–25, 1995.
- [43] J. Dongarra, M. A. Heroux, and P. Luszczyk, "HPCG benchmark: A new metric for ranking high performance computing systems," Electrical Eng. Comput. Sci. Dept., Univ. Tennessee, Tech. Rep. UT-EECS-15-736, 2015.
- [44] G. Blake, R. G. Dreslinski, T. Mudge, and K. Flautner, "Evolution of thread-level parallelism in desktop applications," *SIGARCH Comput. Archit. News*, vol. 38, no. 3, pp. 302–313, 2010.



Janaina Schwarzrock received the BS degree in computer science from UDESC, Brazil, in 2013, and the MSc degree from UFRGS, Brazil, in 2018. Currently, she is working toward the PhD degree at UFRGS. Her primary research interests include the TLP exploitation in NUMA multi-core systems, dynamic power management techniques, and thread- and data-mapping.



Charles C. de Oliveira received the computer engineering degree from UnB, Brazil, in 2016, and the MSc degree from UFRGS, Brazil, in 2019. His primary research interests include the TLP exploitation in multicores systems and dynamic power management techniques.



Marcus Ritt received the master's degree in computer science from Eberhard-Karls-Universität in Tübingen, in 1997, and the PhD degree in computer science, summa cum laude from Eberhard-Karls-Universität in Tübingen, in 2003. Currently, he is a professor with the Theoretical Informatics Department, Informatics Institute of UFRGS. His current research interests include exact, approximative, and heuristic algorithms for combinatorial optimization problems. For more information, please visit: www.inf.ufrgs.br/mrprritt.



Arthur F. Lorenzon received the BS degree in computer science from UNIPAMPA, Brazil, in 2012, and the MSc and Dr degrees from UFRGS, Brazil, in 2014 and 2018, respectively. Currently, he is a professor with UNIPAMPA. His research interests include parallelism exploitation aiming energy efficiency and the development of approaches to automate the TLP exploitation. For more information, please visit www.inf.ufrgs.br/aflorenzon.



Antonio Carlos S. Beck received the Dr degree from Universidade Federal do Rio Grande do Sul (UFRGS), Brazil, in 2008. Currently, he is an associate professor with the Applied Informatics Department, Informatics Institute of UFRGS. His primary research interests include computer architectures and embedded systems design, and focusing on power consumption. For more information, please visit: www.inf.ufrgs.br/caco.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.